

POLITECNICO DI TORINO



Department of Automation and Informatics (DAUIN)

Master Degree in Mechatronic Engineering

2024/2025

Optimization and Simulation for Industrial Automation

Prof. Alberto Tarable

Prof. Marco Ghirardi

Final project

STUDENTS:

Calderaro Beatrice (333775);

Fornasier Francesca (338688);

Tagarelli Valentina (322331);

Serraglio Giovanni (333770);

1 Emergency scenario

1. Implement the model in Simulink, with parameters m1, m2 and m3.

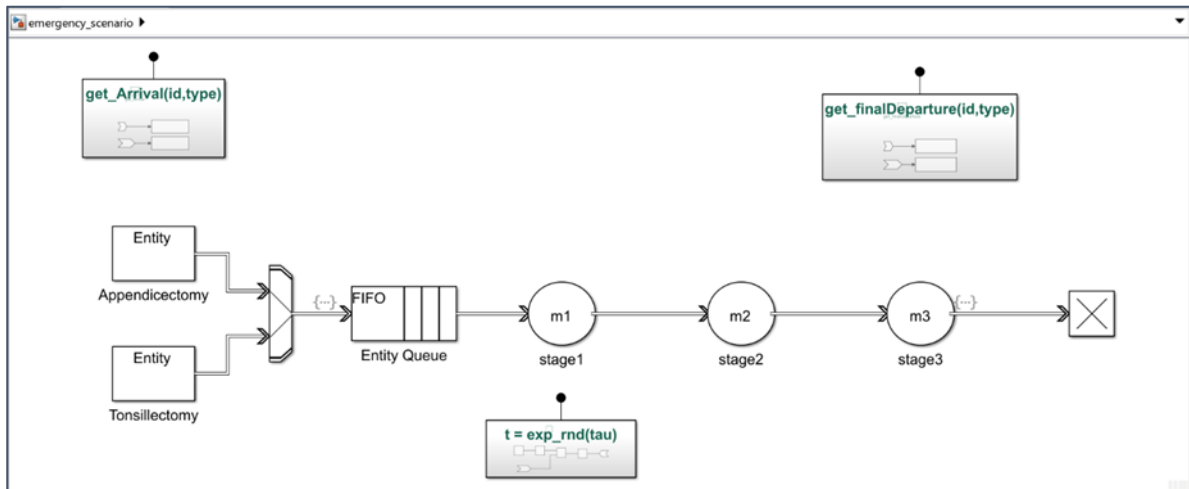


Figure 1. Emergency Scenario Simulink Model

The implemented Simulink model provides an accurate representation of the patient flow within a small surgical unit, capturing the entire process from admission to discharge. The unit is designed to handle two types of surgeries, appendectomy and tonsillectomy, both requiring three distinct stages. Each stage consists of a set of identical rooms, each capable of accommodating a single patient at a time. When a patient reaches a stage, they are assigned to any available room within that stage. If all rooms at a given stage are occupied, different queuing mechanisms apply depending on the stage.

The first stage, anaesthesia, follows an $M/M/m$ queuing model. This means that patients arriving at this stage enter a practically unlimited waiting room if no anaesthesia room is immediately available. They remain there until one of the $m1$ anaesthesia rooms becomes free. This assumption reflects a real-world hospital scenario where a waiting area is available for incoming patients before they begin treatment.

The second and third stages, corresponding to surgery and recovery, operate under an $M/M/m/m$ model. This means that the number of rooms at these stages imposes a strict limit on patient flow. A patient who finishes anaesthesia and is ready for surgery moves to any of the $m2$ available operating rooms. However, if all $m2$ rooms are occupied, the patient must remain in their anaesthesia room until a surgical room becomes free. The same rule applies when transitioning from surgery to recovery: patients cannot proceed unless one of the $m3$ recovery rooms is available. Unlike the first stage, no additional waiting area exists for these stages, forcing patients to remain in their current room until they can advance.

This structure accurately models hospital constraints by ensuring that each stage has a limited number of available resources, affecting overall patient flow and potential bottlenecks. The model allows for the analysis of resource allocation and system efficiency while identifying where congestion occurs based on room availability and patient demand.

Entity Generator

The model begins with the Entity Generator block, responsible for simulating patient arrivals. To realistically represent this process, arrivals are modeled as a Poisson process. This means that the time between two consecutive arrivals is a random variable following an *exponential* distribution, with a mean equal to $1/\lambda$, where λ represents the average patient arrival rate per hour.

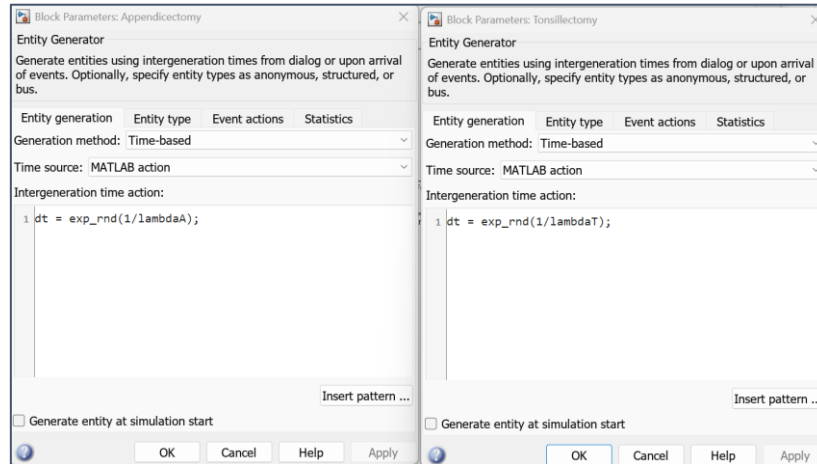


Figure 2. Entity Generator Intergeneration time action

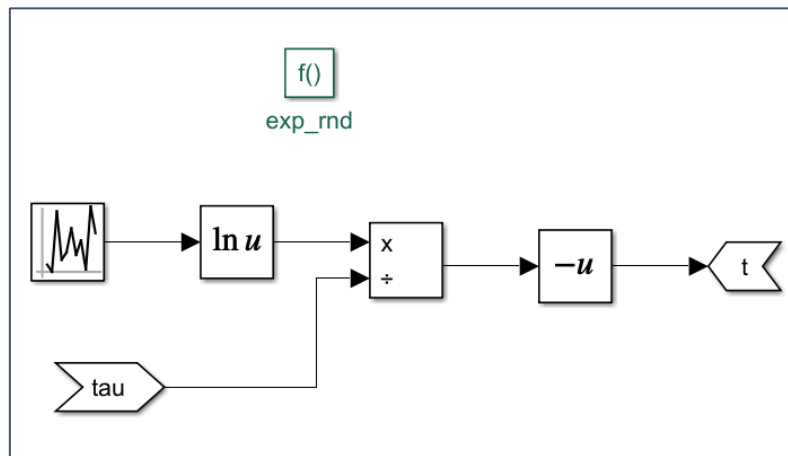


Figure 3. Exponential Distribution Simulink Function

In this model, we have set up two distinct entity generators: one for patients requiring an appendectomy and another for those undergoing a tonsillectomy. Each generated entity is assigned a type attribute, where type = 1 corresponds to an appendectomy and type = 2 to a tonsillectomy. Additionally, the Entity Type section is used to assign each entity the corresponding τ values for the different surgical stages, based on the procedure type. The actual numerical values of the average service times are dynamically assigned at runtime through the MATLAB script.

Attribute Name	Attribute Initial Value
1 tau_1	tau_A_1
2 tau_2	tau_A_2
3 tau_3	tau_A_3
4 type	1

Attribute Name	Attribute Initial Value
1 tau_1	tau_T_1
2 tau_2	tau_T_2
3 tau_3	tau_T_3
4 type	2

Figure 4. Entity Generator Attributes for Appendicectomy and Tonsillectomy

Entity Queue

We have included an Entity Queue, which functions as a waiting line with infinite capacity. This queue is configured with a *FIFO* (First In, First Out) policy, meaning that patients are served in the order they arrive. If there are no available anaesthesia rooms in a phase, patients wait here until a room becomes free.

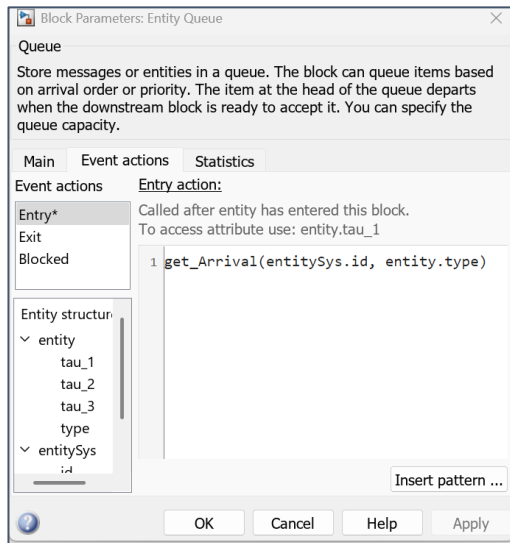


Figure 5. Entity Queue Entry action

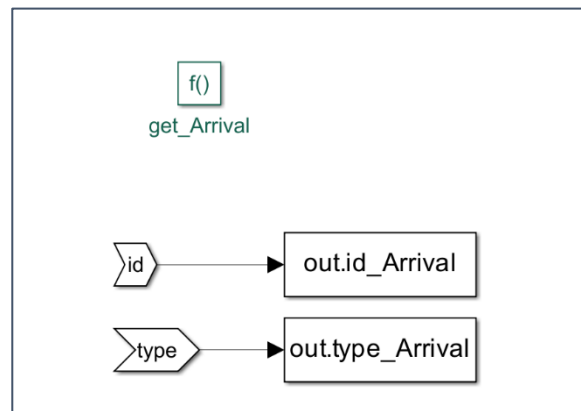


Figure 6. get_Arrival Simulink Function

When an entity enters the queue, the Entry Action triggers the `get_Arrival` function, which extracts the entity's ID and type. These attributes are then assigned to `out.id_Arrival` and `out.type_Arrival`, sending the data to Simulink's workspace for further analysis.

This mechanism ensures that every arriving entity is logged, allowing for accurate tracking of patient flow. The recorded data is later used in MATLAB for post-simulation processing, such as calculating flow times.

Entity Server

The operational phases of the system, namely anaesthesia, operation and recovery, are modelled using the Entity Server blocks. Each block is configured to simulate the treatment time required to complete each phase.

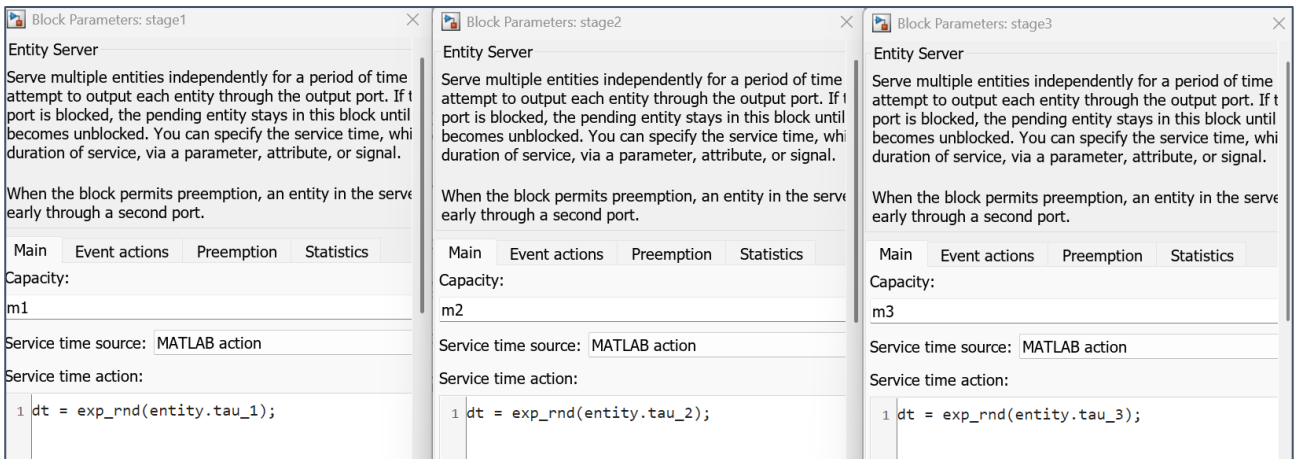


Figure 7. Entity Server Service time action

The service time, for all of them, is generated randomly using the exponential distribution shown in *Figure 3*, with the appropriate tau parameters. The average treatment time varies based on both the patient type and the specific phase, reflecting the different needs of each type of procedure.

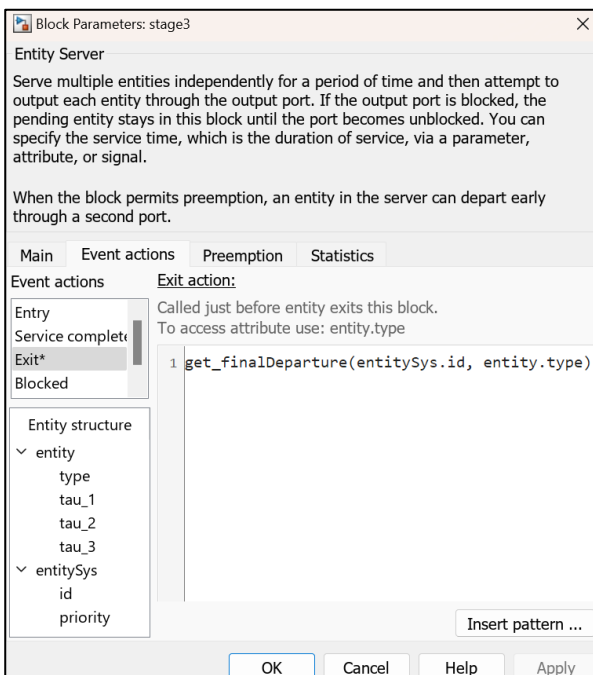


Figure 8. Entity Server Exit action

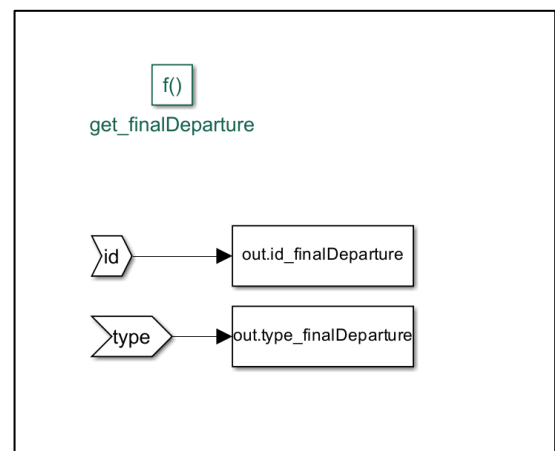


Figure 9. get_finalDeparture Simulink Function

When an entity completes its final stage, the Exit Action in the Entity Server Block at Stage 3, executes the get_finalDeparture Simulink function, capturing the entity's ID and type and storing them in out.id_finalDeparture and out.type_finalDeparture.

This function works in conjunction with get_Arrival, which logs the initial entry of each entity into the system. By combining both arrival and departure data, the MATLAB script can accurately compute each patient's flow time, enabling a precise evaluation.

2. For given values of the parameters, compute the average flow time $\bar{S}_A$ and $\bar{S}_T$ of the two types of patients separately (i.e., the time from the arrival of the patient to the hospital discharge.)

The implemented algorithm aims to calculate the average flow time for two categories of patients: those undergoing appendectomy (SA) and those undergoing tonsillectomy (ST). The flow time represents the time a patient spends within the system, from their arrival to discharge, that is, the completion of their treatment.

```
%% ===== EMERGENCY SCENARIO parameters =====

% --- Service times (hours) for Appendectomy
tau_A_1 = 1;    % Stage 1
tau_A_2 = 2;    % Stage 2
tau_A_3 = 0.5;  % Stage 3

% --- Service times (hours) for Tonsillectomy
tau_T_1 = 0.5;
tau_T_2 = 2;
tau_T_3 = 1.5;

% --- Arrival rates
lambdaA = 1;    % patients/hour (Appendectomy)
lambdaT = 2;    % patients/hour (Tonsillectomy)
```

Figure 10. Emergency Scenario Parameters Definition - MATLAB code

The process begins with the definition of the initial parameters, which include the service times for each type of procedure and the arrival rates of the patients. The service times for appendectomy (tau_A_1, tau_A_2, tau_A_3) and for tonsillectomy (tau_T_1, tau_T_2, tau_T_3) are specified for each stage of the treatment (anesthesia, surgery, recovery). Additionally, the arrival rates for each type of patient are defined: lambdaA for appendectomy patients and lambdaT for tonsillectomy patients.

```
%% ===== PART 1: SIMULATION (ARBITRARY VALUES) =====

% Choose an arbitrary set of rooms
v_m1 = [1 2 3 5 10 50]; % rooms in stage 1
v_m2 = [1 2 3 4 6 50]; % rooms in stage 2
v_m3 = [1 3 5 7 10 50]; % rooms in stage 3

for i = 1: length(v_m1)
    m1 = v_m1(i); m2 = v_m2(i); m3 = v_m3(i);

    % Open the Simulink model (make sure "emergency_scenario.slx" is on the path)
    open_system('emergency_scenario');

    % Run the simulation
    simOut = sim('emergency_scenario', 'StopTime', '10000');
    [mean_SA, mean_ST, mean_TF] = computeFlowTimes(simOut);

    fprintf('----- SIMULATION RESULTS (ARBITRARY VALUES) -----\n');
    fprintf('m1 = %d, m2 = %d, m3 = %d\n', m1, m2, m3);
    fprintf('Average flow time (Appendectomy patients) SA: %.2f hours\n', mean_SA);
    fprintf('Average flow time (Tonsillectomy patients) ST: %.2f hours\n', mean_ST);
    fprintf('Total Average flow time (all patients): %.2f hours\n', mean_TF);
end
```

Figure 11. Emergency Scenario Part 1: Simulation (Arbitrary Values of m1, m2, m3) - MATLAB code

The main goal of this first part is to determine how the number of rooms in each stage affects the flow time of patients through the system. By running multiple simulations in Simulink and systematically adjusting the number of rooms, the script provides insights into how different configurations influence the Emergency Scenario efficiency.

The first step in the script is defining a set of arbitrary room allocations for each stage. Three vectors, `v_m1`, `v_m2`, and `v_m3`, store different values representing the number of rooms available in stage 1, stage 2, and stage 3, respectively. These values range from very limited capacities, such as a single room, to highly resourced scenarios with up to 50 rooms. This range allows for the analysis of how both small and large-scale setups perform under the same patient flow conditions.

With the room configurations defined, the script enters a loop where it iterates over each combination of values. During each iteration, the values for `m1`, `m2`, and `m3` are extracted from the predefined vectors, representing a particular setup to be tested.

Once a configuration is selected, the Simulink model is opened and executed. The model runs for a total of 10,000 hours, a sufficiently long simulation time to capture steady-state performance. By running the simulation over an extended period, the results are not biased by initial transient effects, ensuring a more reliable assessment of the Emergency Scenario's operational efficiency. After the simulation is completed, the script extracts the relevant flow time data using the function `computeFlowTimes(simOut)`. This function calculates three key metrics: the average flow time for appendectomy patients (`mean_SA`), the average flow time for tonsillectomy patients (`mean_ST`), and the total average flow time across all patients (`mean_TF`).

Finally, the script prints the results for each iteration. It displays the specific room allocation used, along with the corresponding flow times for appendectomy and tonsillectomy patients, as well as the overall system-wide average.

```

function [mean_SA, mean_ST, mean_TF] = computeFlowTimes(simOut)
    % Extract arrival/departure data
    arrival_data = simOut.id_Arrival.Data;
    arrival_time = simOut.id_Arrival.Time;
    departure_data = simOut.id_finalDeparture.Data;
    departure_time = simOut.id_finalDeparture.Time;

    arrival_type = simOut.type_Arrival.Data;
    departure_type = simOut.type_finalDeparture.Data;

    % Check for incomplete entities
    missing_ids = setdiff(arrival_data, departure_data);
    if ~isempty(missing_ids)
        warning('There are %d entities that have not completed the process.', length(missing_ids));
    end

    % Compute flow times
    flow_time_A = [];
    flow_time_T = [];
    totalFlow_time = [];

    for i = 1:length(departure_data)
        id_out = departure_data(i);
        idx = find(arrival_data == id_out, 1);

        if ~isempty(idx)
            flow_time = departure_time(i) - arrival_time(idx);
            totalFlow_time = [totalFlow_time; flow_time];

            if departure_type(i) == 1 % Appendectomy
                flow_time_A = [flow_time_A; flow_time];
            elseif departure_type(i) == 2 % Tonsillectomy
                flow_time_T = [flow_time_T; flow_time];
            end
        end
    end
end

```

Figure 12. computeFlowTimes MATLAB Function

The computeFlowTimes function processes data extracted from the simulation output (simOut) to determine the time patients spend in the hospital system. It retrieves arrival and departure times, along with unique patient IDs, ensuring that each patient who entered the system is accounted for. If any patients are missing, meaning they arrived but never exited, the function issues a warning, alerting that some entities have not completed the process.

To calculate flow times, the function matches each departing patient to their corresponding arrival record using their unique ID. The flow time for each patient is computed by subtracting the arrival time from the departure time, determining the total duration the patient spent in the system. These values are then categorized based on patient type, distinguishing between appendectomy patients, tonsillectomy patients, and a general pool that includes all patients.

```

% Calculate mean flow times
if ~isempty(flow_time_A)
    mean_SA = mean(flow_time_A);
else
    mean_SA = NaN;
end

if ~isempty(flow_time_T)
    mean_ST = mean(flow_time_T);
else
    mean_ST = NaN;
end

if ~isempty(totalFlow_time)
    mean_TF = mean(totalFlow_time);
else
    mean_TF = NaN;
end
end

```

Figure 13. Average Flow Time computation inside computeFlowTimes MATLAB Function

Once the individual flow times have been collected, the function proceeds to calculate the average flow times for each category. If there are recorded flow times for appendectomy patients, the function computes mean_SA as the average of all flow times in that category; otherwise, it assigns a NaN value to indicate the absence of valid data. The same logic applies to tonsillectomy patients, where mean_ST is either the computed mean or NaN if no valid data exists. The function also calculates mean_TF, representing the overall average flow time for all patients combined. If no patients have completed the process, this value is also assigned NaN to maintain consistency in the output.

Results

The following results present the impact of varying the number of rooms at each stage on patient flow times. By analysing different configurations, we can observe how system performance evolves with increasing capacity.

```

Warning: There are 10223 entities that have not completed the process.
----- SIMULATION RESULTS (ARBITRARY VALUES) -----
m1 = 1, m2 = 1, m3 = 1
Average flow time (Appendectomy patients) SA: 3403.54 hours
Average flow time (Tonsillectomy patients) ST: 3388.04 hours
Total Average flow time (all patients): 3398.27 hours
Warning: There are 1888 entities that have not completed the process.
----- SIMULATION RESULTS (ARBITRARY VALUES) -----
m1 = 2, m2 = 2, m3 = 3
Average flow time (Appendectomy patients) SA: 657.54 hours
Average flow time (Tonsillectomy patients) ST: 648.28 hours
Total Average flow time (all patients): 654.44 hours
Warning: There are 2 entities that have not completed the process.
----- SIMULATION RESULTS (ARBITRARY VALUES) -----
m1 = 3, m2 = 3, m3 = 5
Average flow time (Appendectomy patients) SA: 4.26 hours
Average flow time (Tonsillectomy patients) ST: 3.91 hours
Total Average flow time (all patients): 4.14 hours
Warning: There are 3 entities that have not completed the process.

```

Figure 14.1. Command Window Results from Part 1: Simulation (Arbitrary Values of m1, m2, m3)

```

----- SIMULATION RESULTS (ARBITRARY VALUES) -----
m1 = 5, m2 = 4, m3 = 7
Average flow time (Appendectomy patients) SA: 3.52 hours
Average flow time (Tonsillectomy patients) ST: 3.22 hours
Total Average flow time (all patients): 3.42 hours
Warning: There are 3 entities that have not completed the process.
----- SIMULATION RESULTS (ARBITRARY VALUES) -----
m1 = 10, m2 = 6, m3 = 10
Average flow time (Appendectomy patients) SA: 3.51 hours
Average flow time (Tonsillectomy patients) ST: 3.16 hours
Total Average flow time (all patients): 3.39 hours
Warning: There are 7 entities that have not completed the process.
----- SIMULATION RESULTS (ARBITRARY VALUES) -----
m1 = 50, m2 = 50, m3 = 50
Average flow time (Appendectomy patients) SA: 3.51 hours
Average flow time (Tonsillectomy patients) ST: 3.12 hours
Total Average flow time (all patients): 3.38 hours

```

Figure 14.2. Command Window Results from Part 1: Simulation (Arbitrary Values of m_1 , m_2 , m_3)

Analysis of Patient Flow and Room Allocation

From the analysis of the simulation results, a significant pattern emerges regarding the *average flow time* for appendectomy (SA) and tonsillectomy (ST) patients. Initially, with a reduced number of rooms (e.g., $m_1 = 1$, $m_2 = 1$, $m_3 = 1$), the flow time is very high (3403.54 hours for appendectomy patients and 3388.04 hours for tonsillectomy patients). However, as the number of rooms increases (e.g., $m_1 = 5$, $m_2 = 4$, $m_3 = 7$), the average flow time decreases drastically, stabilizing around 3-4 hours for both patient types.

Once a certain threshold in the number of rooms is reached, as in the cases with $m_1 = 10$, $m_2 = 6$, $m_3 = 10$ or $m_1 = 50$, $m_2 = 50$, $m_3 = 50$, the flow time does not undergo significant further changes. In fact, in both cases, the flow times stabilize around similar values: 3.51 hours for appendectomy patients and 3.12 hours for tonsillectomy patients. This suggests that, beyond a certain amount of resources, the system reaches a point of saturation, where increasing the number of rooms does not produce tangible improvements in flow times.

Therefore, the analysis implies that increasing the number of rooms, once an adequate threshold to meet demand is reached, becomes redundant and does not justify the additional costs.

3. Supposing that one stage-1 room costs 10 k€, one stage-2 room costs 50 k€ and one stage-3 room costs 2 k€, find the minimum-cost set of parameters that allow to have $\bar{S} \leq 20$ hours, where $\bar{S}$ is the average flow time for all patients.

To achieve this objective, an algorithm was implemented that combines an exhaustive search of possible room combinations with an integer linear programming (LP) solver. Below, the reasoning behind the algorithm and the results obtained are explained.

Definition of Variables and Costs

The algorithm begins by defining the key parameters, including the search ranges for the number of rooms in each treatment stage and their corresponding costs.

```
%% ===== PART 2: EXHAUSTIVE SEARCH + LP SOLVER =====

% Threshold on the overall average flow time
Smax = 20;

% Define search ranges for (m1, m2, m3)
range_m1 = 1:10;
range_m2 = 1:10;
range_m3 = 1:10;

% Define costs (k€) for each room
cost_stage1 = 10;
cost_stage2 = 50;
cost_stage3 = 2;

% Arrays to collect feasible combinations and results
Comb = []; % (m1, m2, m3)
Svals = []; % average flow time (all patients)
Costvals = []; % cost for each combination
```

Figure 15. Emergency Scenario Part 2: Exhaustive Search + LP Solver Parameters Definition – MATLAB code

Exhaustive Search of Combinations

```
% Loop over all (m1, m2, m3)
for m1 = range_m1
    for m2 = range_m2
        for m3 = range_m3

            % Run the simulation for a sufficiently large StopTime
            simOut = sim('emergency_scenario','StopTime','10000');

            % Calculate the overall average flow time (all patients)
            meanS_all = computeTotalFlowTime(simOut);

            % Keep this combination if meanS_all <= Smax
            if meanS_all <= Smax
                cost = cost_stage1*m1 + cost_stage2*m2 + cost_stage3*m3;

                Comb = [Comb; m1, m2, m3];
                Svals = [Svals; meanS_all];
                Costvals = [Costvals; cost];
            end
        end
    end
end
```

Figure 16. Emergency Scenario Part 2: Exhaustive Search

The first step was to explore all possible room combinations for the three stages. The combinations were generated by considering a range from 1 to 10 rooms for each stage (m_1 , m_2 , m_3). For each combination, a simulation of the Simulink model was executed for a period of 10,000 hours to calculate the total average flow time ($\bar{S}$), which is computed using a function called *computeTotalFlowTime*, detailed in Figure 18.

During the simulation, patient arrival and departure data were used to compute the total average flow time. Combinations that met the constraint $\bar{S} \leq 20$ hours were retained for further analysis along with their associated total cost.

```
% Check how many feasible combinations we have
nFeas = size(Comb,1);
if nFeas == 0
    warning('No combination found with average flow time <= %.1f hours.', Smax);
    return;
end
```

Figure 17. Emergency Scenario Part 2: Check feasible

With this check, the code verifies whether there are valid room combinations that satisfy the constraint on the average flow time. If no valid combinations are found (i.e., if *nFeas* is 0), a warning is issued, and the process is terminated.

```
function meanS_all = computeTotalFlowTime(simOut)

    arrival_data = simOut.id_Arrival.Data;
    arrival_time = simOut.id_Arrival.Time;

    departure_data = simOut.id_finalDeparture.Data;
    departure_time = simOut.id_finalDeparture.Time;

    % Flow times array
    flow_times = zeros(length(departure_data),1);
    for iEnt = 1:length(departure_data)
        id_out = departure_data(iEnt);
        idx_in = find(arrival_data == id_out, 1);
        if ~isempty(idx_in)
            flow_times(iEnt) = departure_time(iEnt) - arrival_time(idx_in);
        end
    end

    meanS_all = mean(flow_times);
end
```

Figure 18. *computeTotalFlowTimes* MATLAB Function

The *computeTotalFlowTime* function is designed to calculate the average flow time of all patients in the system, using the data related to arrivals and departures obtained from the simulation. To do this, the function accesses the arrival and departure data of the patients, which are provided by the simulation through two variables: *arrival_data* and *departure_data*. These data contain the patient IDs and the corresponding arrival and departure times, respectively.

The function begins by extracting the arrival and departure data. Then, for each patient, it searches for the matching arrival ID in the arrival list and, once found, calculates the time spent in the system, which is the difference between the departure time and the arrival time. This is done for every patient who has both an arrival and a departure recorded in the system. The flow times are stored in an array, *flow_times*, which collects the stay durations of all patients in the system.

Once all the flow times have been calculated, the function computes the average of these times using the *mean()* function, thus returning the average flow time for the entire group of patients. This value represents the average time a patient spends in the system, from entry to discharge.

Optimization with LP Solver

```
% Build the LP model
% x(i) = 1 if we pick combination i, 0 otherwise

x = optimvar('x', nFeas, 'Type', 'integer', 'LowerBound', 0, 'UpperBound', 1);

% Objective: minimize sum(cost * x)
obj = sum(Costvals .* x);

mProblem = optimproblem;
mProblem.Objective = obj;

% Constraint: exactly one combination must be chosen
mProblem.Constraints.OneCombo = sum(x) == 1;

% Solve with intlinprog
[Sol, fval] = solve(mProblem, 'solver', 'intlinprog');

% Retrieve solution
bestCost = fval;
chosenIdx = find(Sol.x);

if length(chosenIdx) ~= 1
    warning('The solver did not return a single combination (exitflag = %d).', exitflag);
else
    best_m1 = Comb(chosenIdx,1);
    best_m2 = Comb(chosenIdx,2);
    best_m3 = Comb(chosenIdx,3);
    bestS    = Svals(chosenIdx);

    fprintf('\n----- OPTIMIZATION RESULTS (LP SOLVER) ----- \n');
    fprintf('Selected Combination: m1=%d, m2=%d, m3=%d\n', best_m1, best_m2, best_m3);
    fprintf('Total Average flow time (all patients) = %.2f hours\n', bestS);
    fprintf('Total Cost = %.0f k€\n', bestCost);
end
```

Figure 19. Emergency Scenario Part 2: LP Solver – MATLAB code

After identifying the valid configurations, the problem is transformed into an Integer Linear Programming (LP Solver) model. A binary vector $x(i)$ is defined, where each element indicates whether a specific room configuration is selected (1) or not (0). The objective is to minimize the total cost, represented as the product of the cost of each configuration (*Costvals*) and the vector x .

The main constraint ensures that exactly one configuration is selected, guaranteeing that only one room setup is active. The *intlinprog* solver is used to solve the optimization problem and returns

the optimal solution, including the index of the chosen configuration (*chosenIdx*), the total cost (*fval*), and the corresponding values of *m1*, *m2*, *m3*, and the average flow time (*bestS*). This solution represents the room configuration that offers the best trade-off between cost and efficiency while satisfying the flow time constraint. Finally, the results are printed, displaying the selected combination of rooms, the total average flow time, and the total cost.

Analysis of the Trade-off Between Costs and Flow Time

An interesting insight from the analysis is that beyond a certain threshold of resources, adding more rooms leads to a significant increase in costs without yielding noticeable improvements in average flow time.

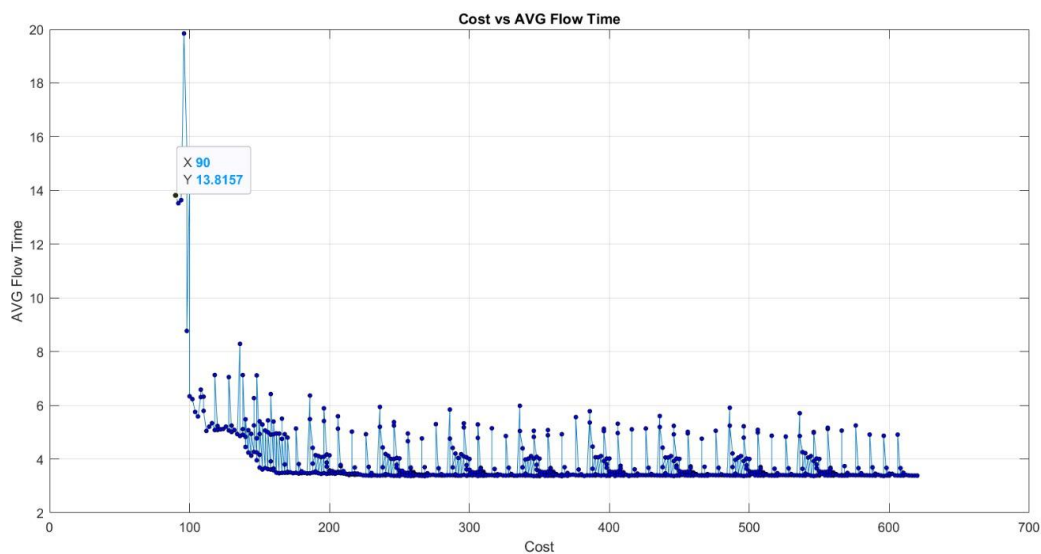


Figure 20. Cost – AVG Flow Time Plot

By examining the results, it becomes evident that increasing the number of rooms initially results in a substantial reduction in flow time. However, beyond a certain point, the effect stabilizes. This phenomenon occurs because, once the system has sufficient capacity to handle patient flow without significant delays, additional rooms do not enhance performance but merely increase costs.

This finding highlights the importance of striking a balance between available resources and costs: investing in too many rooms becomes inefficient and does not provide tangible benefits in terms of reducing patient waiting times.

4. Conclusion

The results obtained from the optimization using the *intlinprog* solver show that the optimal room configuration is **m1=3, m2=1, and m3=5**. This setup ensures a total average flow time of **13.82 hours**, well below the 20-hour constraint, with a **total cost of 90 k€**.

```
Command Window

Solving problem using intlinprog.
LP:          Optimal objective value is 90.000000.

|
Optimal solution found.

Intlinprog stopped at the root node because the objective value is within a gap tolerance of the optimal value,
options.AbsoluteGapTolerance = 0. The intcon variables are integer within tolerance,
options.IntegerTolerance = 1e-05.

----- OPTIMIZATION RESULTS (LP SOLVER) -----
Selected Combination: m1=3, m2=1, m3=5
Total Average flow time (all patients) = 13.82 hours
Total Cost = 90 k€
fx >> |
```

Figure 21. Command Window Results from Part 2: Exhaustive Search +LP Solver

Results Commentary

1.Operational Efficiency: The average flow time of **13.82 hours** indicates that the system is highly efficient, with a well-balanced allocation of resources across the three phases (anaesthesia, surgery, and recovery). This result demonstrates that patient flow can be managed optimally without the need for additional investments.

2.Optimized Cost: The total cost of **90 k€** is the minimum required to meet the flow time constraint. This finding is particularly significant because it shows that, with a well-planned configuration, costs can be minimized without compromising system efficiency.

3.Room Allocation: The selection of **3 rooms for anaesthesia (m1), 1 room for surgery (m2), and 5 rooms for recovery (m3)** reflects an intelligent resource allocation strategy. The greater emphasis on recovery (m3) is justified by the fact that this phase has a lower cost (2 k€ per room) compared to surgery (50 k€ per room), allowing for an increased number of rooms without significantly impacting total costs.

In summary, the results demonstrate that it is possible to achieve a balance between costs and operational efficiency, ensuring high-quality service for patients.

2 Hospital scenario

1. Implement the model in Simulink, with $m1 = 1$, $m2 = 1$, $m3 = 1$ and $N = 20$

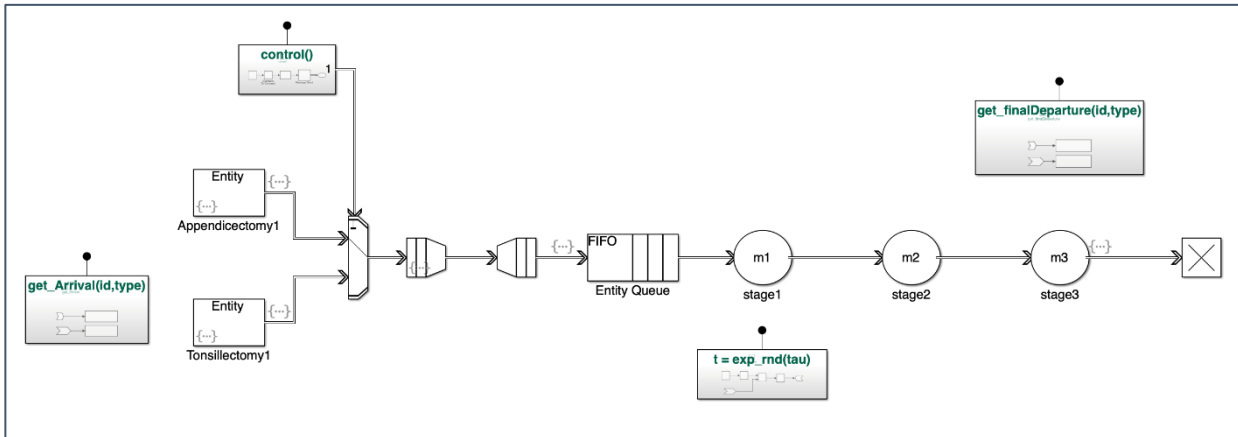


Figure 22. Hospital Scenario Simulink Model

The implemented Simulink model simulates patient flow in a small surgical unit, managing a total of 20 patients, evenly divided between 10 appendectomy and 10 tonsillectomy cases. All patients have already been triaged, and their service times for each of the three treatment stages are predetermined at time 0.

The model begins with two separate entity generation blocks: one for appendectomy patients and the other for tonsillectomy patients. These entities enter a switch controlled by a control block, which uses a random distribution logic to select the type of patient entering the system. This random selection creates variability in the order of patients processed, simulating unpredictable real-world patient arrivals.

After passing through the switch, patients are grouped using a batch creation mechanism, organizing them temporarily before they proceed further. A batch splitter then separates these grouped entities into individual patients to ensure independent processing at each subsequent stage.

Once separated, the patients enter a FIFO (First-In-First-Out) queue that preserves the order of arrival within the queue itself. The queue feeds into three sequential treatment stages: anesthesia (m1), surgery (m2), and recovery (m3). Each stage has only one available room, meaning that only one patient can be treated at a time per stage.

Finally, for this scenario we have to evaluate and compare the patients flow time (from their admission to the completion of treatment) with a random patient ordering and with an optimized ordering.

Entity Generator

In this scenario we need to generate all the entities at time 0, 10 for each type of operation. We use the Entity Generator block with the intergeneration time defined with a Matlab action. Two variables entityCounter_A and igt_A are defined as persistent in order to retain their values between different executions. If the counter variable is empty, we define a vector of $N/2 + 1$ intergeneration times and set the counter to 1; each time the function is executed, dt is assigned

the value from igt_A at the position indicated by entityCounter_A and the counter is incremented. This way we generate the first $N/2$ entities at time zero, after that dt assumes the value infinite. The same function is implemented in the Entity Generator block for Tonsillectomy, with variables entityCounter_T and igt_T.

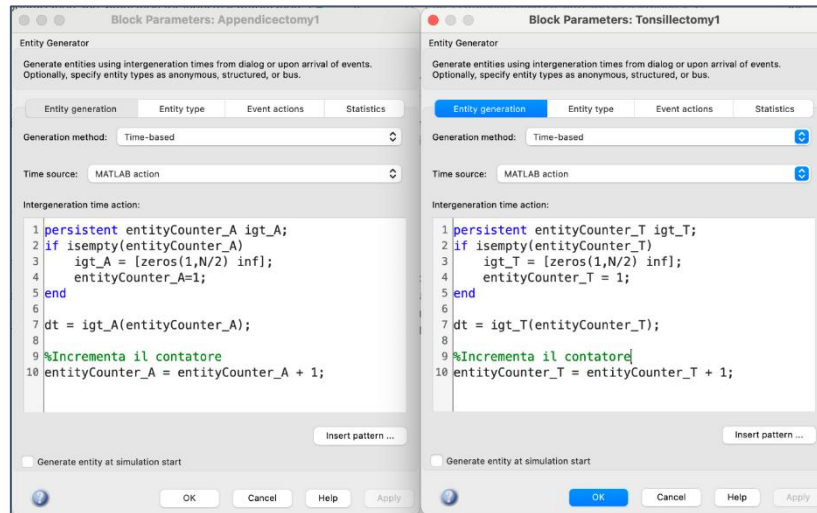


Figure 23. Entity Generator Intergeneration Time action

As in the emergency scenario, in the Entity Type section the entities are characterised by some attributes that make the model able to distinguish between the two types of operations and determine which will be the correct service time required for each specific stage. Here the attributes are initialised with the values defined in the hospital_driver MATLAB script.

In the Event actions section, the attributes tau_1, tau_2, and tau_3 are assigned new values using an exponential distribution. This represents the processing times for each stage of the surgery, ensuring variability in service duration. Since patients have already undergone triage, their service times must be determined at the moment of entity generation (time 0), allowing the system to process each case efficiently based on predefined distributions.

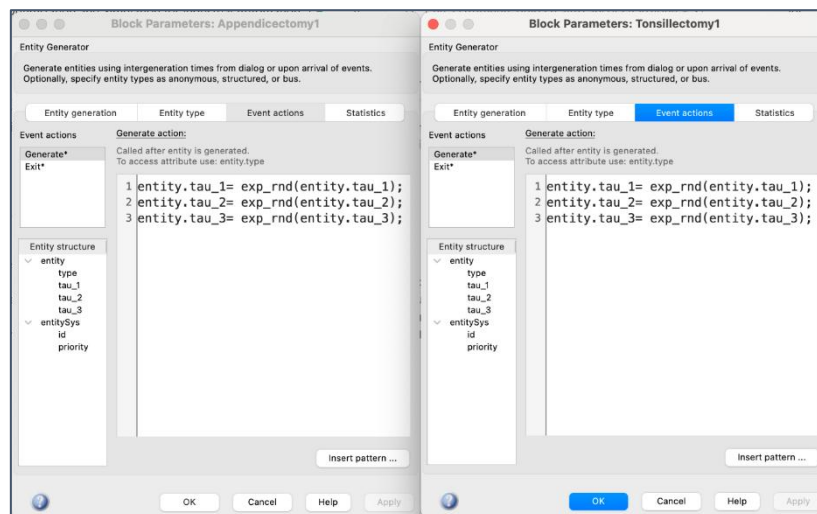


Figure 24. Entity Generator Generate action

Before the entities exit the block, the function `control()` is invoked to properly use the Entity Input Switch block.

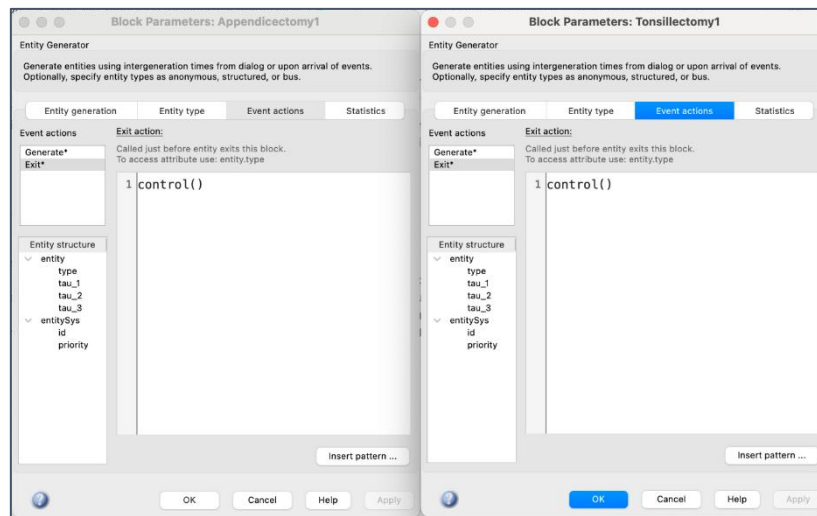


Figure 25. Entity Generator Exit action

Entity Input Switch

To make the entities generated from both sources converge into a single path we included an Entity Input Switch block, with switching criterion determined by the Simulink function `control()` connected to the control port. This way we can achieve a random patients ordering, necessary for the first part of the simulation.

Simulink function `control()`

As previously mentioned, this function is used to decide which input port to activate.

We use a Uniform Random Number block generating a number in the range $[0, 1]$, that is then compared to the value 0.5 using the Compare to constant block. Since the output is 1 or 0 depending on whether the condition in the previous block is satisfied or not, and the switch input ports are addressed starting from 1, we increment the obtained value by one and send it to the output through a Message Send.

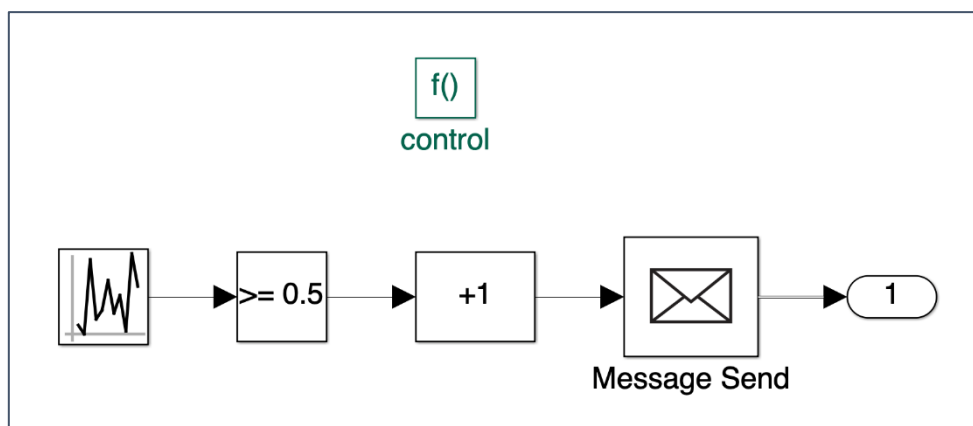


Figure 26. `control()` Simulink Function

Entity Queue

The entities exiting from the switch converge into an Entity Queue with capacity set to N=20 as we know a priori the number of entities. The queue is managed with First In - First Out policy, meaning that patients are served in the same order they arrive.

This works as a waiting room where the patients stay until a stage-1 room is available.

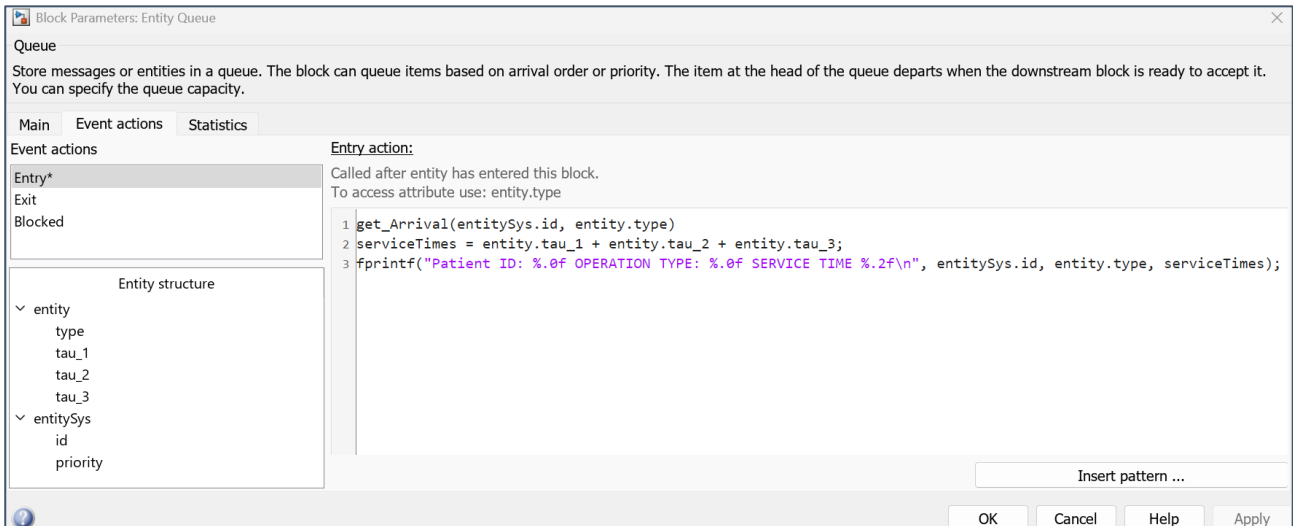


Figure 27. Entity Queue Entry action

When an entity enters the queue, the Entry Action is triggered, calling the `get_Arrival` function (Figure 6), which records the entity's ID and type. This data is stored in Simulink's workspace for monitoring and analysis. The entity's service times are computed by summing the durations of the first, second and third stages (`tau_1`, `tau_2` and `tau_3`), reflecting the total time the patient will spend in treatment.

Additionally, the action outputs a message via the `fprintf` function to the console. This message includes the patient ID, operation type, and calculated service time, providing real-time tracking information during the simulation.

Entity Server

The three stages the patients have to go through are modelled with three Entity Server blocks. The service time for each server is retrieved from the corresponding `tau` attribute of the entity being served, as in the Emergency Scenario (Figure 7).

As in Emergency Scenarios, the Exit Action at Stage 3 (Figure 8) triggers the `get_finalDeparture` (Figure 9) function, logging the entity's ID and type. Together with `get_Arrival`, this enables MATLAB to calculate flow times and evaluate system performance accurately.

2. Implement an algorithm of optimization for finding an efficient order of operation for the N patients, where the parameter to be optimized is the average flow time S

To simulate the system we need to define the initial parameters, including the means of the exponential distributions used to calculate the service time for each stage of the treatment for appendectomy (τ_{A_1} , τ_{A_2} , τ_{A_3}) and tonsillectomy (τ_{T_1} , τ_{T_2} , τ_{T_3}), the number of patients we have in the system and the number of rooms we can use in each stage.

```
%% ===== HOSPITAL SCENARIO parameters =====  
  
% Mean service time (in hours) for each stage of appendectomy  
% Stage 1, Stage 2, Stage 3  
 $\tau_{A_1}$  = 1;  
 $\tau_{A_2}$  = 2;  
 $\tau_{A_3}$  = 0.5;  
  
% Mean service time (in hours) for each stage of tonsillectomy  
% Stage 1, Stage 2, Stage 3  
 $\tau_{T_1}$  = 0.5;  
 $\tau_{T_2}$  = 2;  
 $\tau_{T_3}$  = 1.5;  
  
% Total number of patients to be simulated  
N = 20;  
  
% Number of available rooms for each stage of the hospital process  
m1 = 1; % Rooms in stage 1  
m2 = 1; % Rooms in stage 2  
m3 = 1; % Rooms in stage 3
```

Figure 28. Hospital Scenario Parameters Definition - MATLAB code

To solve this scheduling problem we implement an optimisation function in the batch generator block, executed after the batch entity is generated.

The aim is to minimise the average flow time, that is the amount of time that patients spend in the system, from the arrival to the completion of their recovery and discharge.

Greedy Heuristic Algorithm for Optimized Scheduling

To find an efficient ordering of patients, we implement a greedy heuristic algorithm, consisting in making at each iteration the locally optimal choice without revisiting or changing previous decisions (backtrack is not possible in greedy algorithms).

In this case we use a Shortest Processing Time First scheduling approach, prioritizing patients with the shortest total service time: this ensures shorter operations are completed first, reducing the overall waiting times for following patients.

We define a vector in which for each patient we calculate the total service time as the sum of the required time to complete the three stages, a new array to store the sorted entities and a vector to keep track of which elements have already been selected.

We initialise the variable containing the minimum service time with value infinite so that it will be surely updated during the iteration, and define a variable for the index of the minimum.

For N times we iterate over the elements of the batch to find each time the one with the minimum flow time among those not yet selected, then we mark the resulting entity as selected and add it in the first free position of the sorted array (indicated by j).

After sorting all patients according their service time, we assign to the entity.batch the new sorted batch.

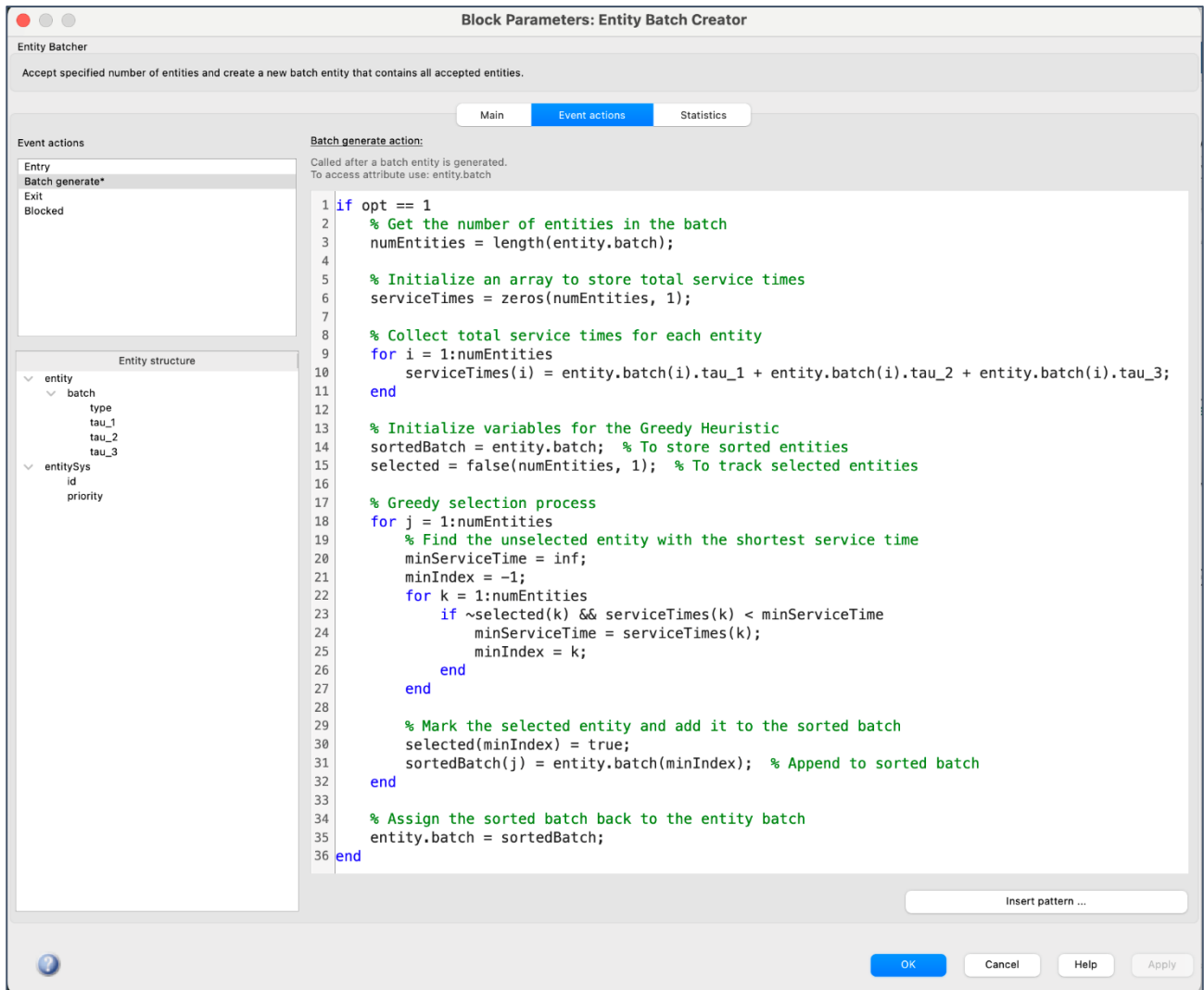


Figure 29. Entity Batch generate action: Greedy Heuristic Algorithm for Optimized Scheduling of the patients

3. Include the optimization algorithm in the model, running it at time 0 and then sending the patients to their operations in an optimized order. Compare the obtained $\bar{S}$ with the one obtained with random ordering of the patients.

To achieve this assignment, the MATLAB script is designed to run the simulation twice, comparing different patient ordering strategies. It accomplishes this by calling the `run_simulation()` function for each scenario and displaying the results using `printf`. The first run measures the average flow time with patients following a random order, while the second evaluates the average flow time after reordering them within the Batch Creator using the optimized Greedy Heuristic Algorithm described in the previous section.

<pre>%% === Compute average flow time using RANDOM patient ordering === opt = 0; mean_S_random = run_simulation(); fprintf("\nSimulation Result: RANDOM ORDER \n"); fprintf("Average Flow Time: %.2f hours\n", mean_S_random); fprintf("-----\n");</pre>	
<pre>%% === Compute average flow time using OPTIMIZED patient ordering (Greedy Heuristic) === opt = 1; mean_S_greedy = run_simulation(); fprintf("\nSimulation Result: GREEDY HEURISTIC OPTIMIZED ORDER \n"); fprintf("Average Flow Time: %.2f hours\n", mean_S_greedy); fprintf("-----\n");</pre>	

Figure 30. AVG Flow Time Random and Optimized ordering - MATLAB code

The variable `opt`, defined in the `hospital_driver` file, is used to distinguish between the two simulation cases: when `opt = 0`, patients follow a random order, and when `opt = 1`, the optimized scheduling algorithm is applied.

run_simulation MATLAB Function

The `run_simulation` function is designed to run the hospital Simulink model and calculate the average flow time for patients.

```

function mean_S = run_simulation()
% Open and simulate the Simulink model for the hospital scenario
open_system('hospital_scenario');
simOut = sim('hospital_scenario');

% Extract arrival and departure data from the simulation results
[arrival_data, arrival_time] = get_arrival_data(simOut);
[departure_data, departure_time] = get_departure_data(simOut);

% Compute flow time for completed entities
flow_time_S = calculate_flow_time(arrival_data, arrival_time, departure_data, departure_time);

% Compute and return the mean flow time
if ~isempty(flow_time_S)
    mean_S = mean(flow_time_S);
else
    mean_S = NaN;
end
end

```

Figure 31. run_simulation MATLAB function

It first opens and executes the hospital_scenario model, capturing the simulation output in the simOut variable. From this output, the function retrieves the entity arrival and departure data, including IDs and timestamps, using the helper functions get_arrival_data and get_departure_data.

```

function [arrival_data, arrival_time] = get_arrival_data(simOut)
% Retrieve the entity arrival IDs and corresponding times from simulation output
arrival_data = simOut.id_Arrival.Data;
arrival_time = simOut.id_Arrival.Time;
end

function [departure_data, departure_time] = get_departure_data(simOut)
% Retrieve the entity departure IDs and corresponding times from simulation output
departure_data = simOut.id_finalDeparture.Data;
departure_time = simOut.id_finalDeparture.Time;
end

```

Figure 32. get_arrival_data, get_departure_data helper functions

The core of the function involves calculating flow times, which represent the duration each entity spends in the system from arrival to departure. The calculate_flow_time function matches each departure entity with its corresponding arrival based on the entity ID. It then computes the flow time by subtracting the arrival time from the departure time. This process is repeated for all completed entities.

```

function flow_time_S = calculate_flow_time(arrival_data, arrival_time, departure_data, departure_time)
% Calculate the flow time for each entity that has completed the hospital process
flow_time_S = [];
for i = 1:length(departure_data)
    id_out = departure_data(i);
    idx = find(arrival_data == id_out, 1); % Find corresponding arrival index
    if ~isempty(idx)
        flow_time = departure_time(i) - arrival_time(idx); % Compute flow time
        flow_time_S = [flow_time_S; flow_time];
    end
end
end

```

Figure 33. calculate_flow_time helper function

Finally, the function calculates and returns the mean flow time from the collected flow times. If no flow times are recorded, it returns NaN.

Random Patients Ordering Results

The simulation output presents patient IDs, operation types, and service times in hours. In this scenario, patients are processed in a random order without optimization, alternating between operation types 1 and 2. The service times vary significantly, with some patients requiring longer durations (e.g., Patient 45 with 10.76 hours) and others much shorter (e.g., Patient 54 with 0.39 hours). This variation highlights the unpredictable nature of the process.

```
Patient ID: 42 OPERATION TYPE: 1 SERVICE TIME 3.82
Patient ID: 43 OPERATION TYPE: 1 SERVICE TIME 7.48
Patient ID: 44 OPERATION TYPE: 1 SERVICE TIME 4.04
Patient ID: 45 OPERATION TYPE: 1 SERVICE TIME 10.76
Patient ID: 46 OPERATION TYPE: 2 SERVICE TIME 1.45
Patient ID: 47 OPERATION TYPE: 2 SERVICE TIME 0.65
Patient ID: 48 OPERATION TYPE: 2 SERVICE TIME 5.57
Patient ID: 49 OPERATION TYPE: 2 SERVICE TIME 0.94
Patient ID: 50 OPERATION TYPE: 1 SERVICE TIME 2.12
Patient ID: 51 OPERATION TYPE: 2 SERVICE TIME 4.40
Patient ID: 52 OPERATION TYPE: 2 SERVICE TIME 1.45
Patient ID: 53 OPERATION TYPE: 1 SERVICE TIME 1.90
Patient ID: 54 OPERATION TYPE: 2 SERVICE TIME 0.39
Patient ID: 55 OPERATION TYPE: 1 SERVICE TIME 2.66
Patient ID: 56 OPERATION TYPE: 2 SERVICE TIME 1.45
Patient ID: 57 OPERATION TYPE: 2 SERVICE TIME 4.29
Patient ID: 58 OPERATION TYPE: 2 SERVICE TIME 1.56
Patient ID: 59 OPERATION TYPE: 1 SERVICE TIME 3.51
Patient ID: 60 OPERATION TYPE: 1 SERVICE TIME 4.22
Patient ID: 61 OPERATION TYPE: 1 SERVICE TIME 1.26

Simulation Result: RANDOM ORDER
Average Flow Time: 25.64 hours
```

Figure 34. Command Window Results from Random Patients Ordering Simulation

At the end of the simulation, the system reports an average flow time of 25.64 hours, representing the average time patients spend in the system from arrival to departure. This result sets a baseline for comparison, demonstrating the inefficiency of processing patients without a structured order. It highlights the potential for significant improvements in system performance through optimized scheduling strategies.

Optimized Patients Ordering Results

This simulation output shows the patient IDs, operation types, and service times, but unlike the previous random order scenario, the Greedy Heuristic algorithm is applied within the Batch Creator to optimize the processing sequence. In this case, entities are split and ordered based on their service times, with patients requiring shorter service times processed earlier. As a result, the distribution of patient IDs no longer reflects the sequence of their initial arrival but instead follows the optimized order designed to reduce waiting times.

```

Patient ID: 42 OPERATION TYPE: 2 SERVICE TIME 0.39
Patient ID: 43 OPERATION TYPE: 2 SERVICE TIME 0.65
Patient ID: 44 OPERATION TYPE: 2 SERVICE TIME 0.94
Patient ID: 45 OPERATION TYPE: 1 SERVICE TIME 1.26
Patient ID: 46 OPERATION TYPE: 2 SERVICE TIME 1.45
Patient ID: 47 OPERATION TYPE: 2 SERVICE TIME 1.45
Patient ID: 48 OPERATION TYPE: 2 SERVICE TIME 1.45
Patient ID: 49 OPERATION TYPE: 2 SERVICE TIME 1.56
Patient ID: 50 OPERATION TYPE: 1 SERVICE TIME 1.90
Patient ID: 51 OPERATION TYPE: 1 SERVICE TIME 2.12
Patient ID: 52 OPERATION TYPE: 1 SERVICE TIME 2.66
Patient ID: 53 OPERATION TYPE: 1 SERVICE TIME 3.51
Patient ID: 54 OPERATION TYPE: 1 SERVICE TIME 3.82
Patient ID: 55 OPERATION TYPE: 1 SERVICE TIME 4.04
Patient ID: 56 OPERATION TYPE: 1 SERVICE TIME 4.22
Patient ID: 57 OPERATION TYPE: 2 SERVICE TIME 4.29
Patient ID: 58 OPERATION TYPE: 2 SERVICE TIME 4.40
Patient ID: 59 OPERATION TYPE: 2 SERVICE TIME 5.57
Patient ID: 60 OPERATION TYPE: 1 SERVICE TIME 7.48
Patient ID: 61 OPERATION TYPE: 1 SERVICE TIME 10.76

Simulation Result: GREEDY HEURISTIC OPTIMIZED ORDER
Average Flow Time: 12.09 hours

```

Figure 35. Command Window Results from Optimized Patients Ordering Simulation

This reordering significantly improves system efficiency, as the average flow time drops to 12.09 hours, less than half the time recorded in the random order scenario. This result highlights the effectiveness of the optimization algorithm in enhancing patient flow and reducing overall delays in the system.

Comparison between Random Order and Optimized Order with Greedy Heuristic Algorithm

Comparison and Analysis

```

***** FINAL COMPARISON *****
UNOPTIMIZED (Random) AVG FLOW TIME: 25.64 hours
OPTIMIZED (Greedy Heuristic) AVG FLOW TIME emergency_scenario : 12.09 hours

```

Figure 36. Command Window Results from Final Comparison between the two Patients Ordering strategies

- 1. Reduction in Flow Time:** The use of the Greedy Heuristic algorithm reduced the average flow time from 25.64 hours to 12.09 hours, demonstrating a significant improvement in system efficiency.
- 2. Resource Optimization:** Processing patients with shorter service times first allows resources to be freed up more quickly, reducing bottlenecks and improving overall patient flow.
- 3. Operational Impact:** This result highlights the importance of optimizing the processing order of patients, especially in high-intensity contexts such as a surgical unit, where every minute saved can have a significant impact on service quality.

4. Conclusion

The comparison between the two scenarios clearly shows that using an optimization algorithm, such as the **Greedy Heuristic**, can lead to substantial improvements in operational efficiency. Reducing the average flow time from **25.64 hours** to **12.09 hours** not only enhances the patient experience but also optimizes the use of available resources, making the system more sustainable and effective. This approach can be extended to other operational contexts to improve flow management and reduce waiting times.